Convolutional Neural Networks

Week 13 — Session 2: Training the filters, and when CNNs win

Practical Image Processing

Lecture 13.2 ▪ 60–70 min

Recap of Session 1

- A **convolutional layer** = convolution with *learned* filters (weight sharing).
- CNN recipe: Conv $\rightarrow$ ReLU $\rightarrow$ Pool, stacked, ending in FC + softmax.
- Our digit net: $8 \times 8 \rightarrow$ two conv blocks $\rightarrow$ 10 scores, $\sim$1,900 parameters.
- We can trace shapes and count params — but the filters are still *random*.

Today's job

Turn random filters into good detectors — **train** the net — and judge honestly when a CNN is worth it.

Roadmap — Session 2

Training a CNN

Results

Deep dive

In-class exercise

Wrap-up

Training a CNN

How does it learn the filters?

1. **Forward pass**: run an image through; get predicted probabilities.
2. **Loss**: measure how wrong (**cross-entropy** between prediction and true label).
3. **Backprop**: compute how each weight affected the loss (the gradient).
4. **Update**: nudge every weight to reduce the loss (**gradient descent**).

Key idea

Repeat over many images and epochs; the filters gradually become good detectors.

The forward pass, concretely



- Data flows left-to-right; the output is 10 probabilities.
- Nothing changes during a forward pass — it is pure prediction.
- Learning happens on the way *back*, once we know how wrong we were.

Softmax with real numbers

Suppose the FC layer outputs logits $z = (2.0,\ 0.5,\ -1.0)$ for three classes.

Exponentiate, then normalize

$e^{2.0} = 7.39, \quad e^{0.5} = 1.65, \quad e^{-1.0} = 0.37$

$\text{sum} = 9.41 \Rightarrow p = (0.79,\ 0.18,\ 0.04)$

- Biggest logit $\rightarrow$ biggest probability; everything stays in $[0, 1]$ and sums to 1.
- Softmax is monotone: it never changes *which* class wins, only the confidence.

Cross-entropy loss, intuitively

- The network outputs a probability for each class. Cross-entropy is low when it puts high probability on the *correct* class.
- Confidently wrong $\rightarrow$ huge loss. Correct and confident $\rightarrow$ near-zero loss.
- Training minimizes the average loss over the data.

Math

Loss for one sample $= -\log(p_{\text{true class}})$. Push $p_{\text{true}} \rightarrow 1$.

Cross-entropy, with real numbers

Same example: predicted $p = (0.79, 0.18, 0.04)$. Loss depends only on the *true* class.

True class	p_{true}	**Loss** $= -\log p_{\text{true}}$
class 0 (right, confident)	0.79	0.24
class 1 (partly right)	0.18	1.71
class 2 (confidently wrong)	0.04	3.22

- The more probability we gave the *true* class, the smaller the loss.
- Being confidently wrong is punished hard — that steep penalty drives learning.

Backprop and gradient descent

- **Backprop** applies the chain rule to get $\partial \text{Loss}/\partial w$ for every weight w.
- The gradient says: "increase this weight $\rightarrow$ loss goes up/down, this much."
- **Gradient descent** then steps every weight the opposite way.

The update rule

$$w \leftarrow w - \eta \frac{\partial \text{Loss}}{\partial w} \qquad (\eta = \text{learning rate})$$

Key idea

Backprop finds the direction; the learning rate sets the step. Repeat millions of times.

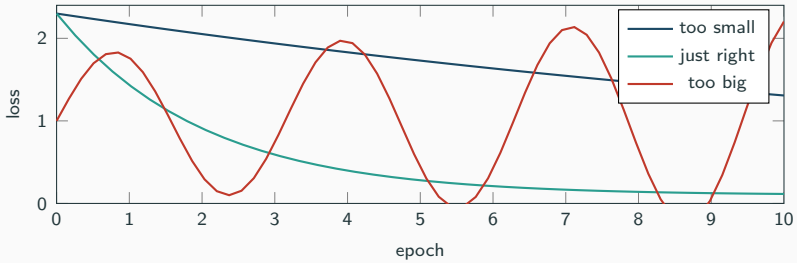
Mini-batches, epochs, learning rate

- **Mini-batch**: update on a small group of images at a time (we used 128) — faster, smoother.
- **Epoch**: one full pass over the training set. We ran 60.
- **Learning rate**: step size of each update. Too big → diverges; too small → crawls.

✕ Pitfall

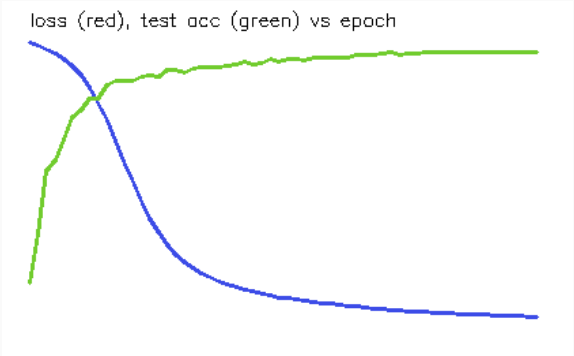
Most training failures are a bad learning rate. If loss explodes or won't move, change it first.

Learning rate: too small, right, too big



Blue crawls, green drops fast and settles, red oscillates and diverges. Tune η first.

Training in action



- Red: training loss falls each epoch.
- Green: test accuracy climbs, then plateaus near 0.97.
- Watching both curves is how you spot under/over-fitting live.

Results

Reading digits — the DocScan payoff



Test digits with the CNN's prediction (green = correct). This is the meter-digit reader DocScan needs.

CNN vs classical — an honest comparison

Method	Test accuracy (digits)
CNN (learned filters)	0.97
SVM on raw pixels	0.99

- On *easy*, clean 8×8 digits, classical ML is already excellent — CNN isn't magic here.
- The CNN's real advantage shows on *hard, large, varied* images — and with **transfer learning** (Week 14).

When do CNNs really win?

- Large, varied datasets where hand features can't capture the variation.
- Problems where the useful features are unknown (the network discovers them).
- Recall Week 12: HOG+SVM plateaued at 48% on look-alike cars — that's where learned features help.

Key idea

Small clean problem $\rightarrow$ classical is fine. Big messy problem $\rightarrow$ deep learning pulls ahead.

Deep dive

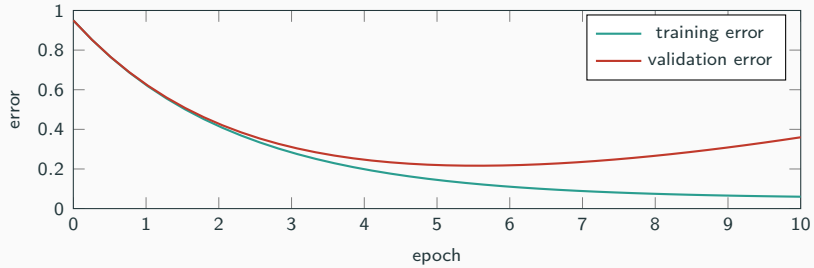
The feature hierarchy

- Layer 1: edges and simple strokes (like the learned filters we saw).
- Layer 2–3: corners, curves, digit parts (loops, junctions).
- Deeper: whole-shape detectors that fire for a specific digit.

Key idea

A CNN builds complexity gradually — simple parts combine into meaningful wholes, all learned.

Overfitting vs underfitting



- Training error keeps dropping; validation error turns *up* — classic **overfitting**.
- Stop where validation is lowest (**early stopping**); if *both* stay high, you **underfit**.

Regularization: keeping CNNs honest

- CNNs have many parameters $\rightarrow$ they can overfit (Week 12's lesson returns).
- **Dropout**: randomly zero some activations during training so the net can't rely on any one.
- **Data augmentation**: random shifts/rotations of training images — free extra data, more invariance.
- **Weight decay**: penalize large weights to keep the model simple.

Data augmentation, concretely

- Take each training digit and make cheap variants: shift 1px, rotate a few degrees, add noise.
- The label stays the same — a shifted "7" is still a 7.
- The net sees more variation → learns invariance for free.

✔ Cheap and effective

On small datasets (like our meter digits), augmentation is often the single biggest accuracy win — before you reach for a bigger model.

Why CNNs need a GPU (and lots of data)

- Millions of multiply-adds per image; a GPU does them massively in parallel.
- Deep nets have millions of parameters — they need many examples to fit without overfitting.
- Our digit net is tiny and trains in seconds; ImageNet-scale nets train for days.

✓ Good practice

Small data + deep net = overfitting. Next week's **transfer learning** fixes exactly this.

A CNN in PyTorch

```
class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.c1 = nn.Conv2d(1, 8, 3, padding=1)
        self.c2 = nn.Conv2d(8, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(16*2*2, 10)
    def forward(self, x):
        x = self.pool(torch.relu(self.c1(x)))
        x = self.pool(torch.relu(self.c2(x)))
        return self.fc(x.flatten(1))
```

Conv $\rightarrow$ ReLU $\rightarrow$ Pool, twice, then a linear classifier. That is the whole digit reader.

Reading the model, line by line

```
self.c1 = nn.Conv2d(1, 8, 3, padding=1)   # 1 -> 8 maps, keep 8x8
self.pool = nn.MaxPool2d(2)               # halve H and W
self.fc = nn.Linear(16*2*2, 10)           # 64 features -> 10 logits
```

- `Conv2d(1,8,3,padding=1)`: matches Session 1's shape table exactly.
- `16*2*2 = 64`: the flattened size after two pools — if this number is wrong, the net won't build.
- `forward` just chains the blocks; `flatten(1)` keeps the batch dimension.

✕ Pitfall

A mismatched `Linear` input size is the #1 first-run CNN error. Compute it from the shape table.

The training loop

```python
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()
for epoch in range(60):
    for xb, yb in train_loader:      # mini-batches of 128
        opt.zero_grad()              # clear old gradients
        logits = net(xb)             # forward pass
        loss = loss_fn(logits, yb)   # cross-entropy
        loss.backward()              # backprop: fill gradients
        opt.step()                   # gradient-descent update
```

- Every line maps to a step from the first training slide: forward, loss, backprop, update.
- `CrossEntropyLoss` does softmax $+ -\log p_{\text{true}}$ in one call.
- Forgetting `zero_grad()` silently accumulates gradients — a classic bug.

In-class exercise

In-class exercise (10 min)

Reason about training

1. The true class has predicted probability 0.5. What is the cross-entropy loss? (use $-\log$)
2. Training loss keeps falling but validation accuracy drops. What is happening, and one fix?
3. Our CNN got 0.97 and the SVM 0.99 on digits. Does that mean CNNs are worse? Explain.
4. You forgot `opt.zero_grad()` in the loop. What goes wrong?

Debrief

- (1) $-\log(0.5) \approx 0.69$ — "half sure of the right answer" costs about 0.69 nats.
- (2) **Overfitting**; fix with early stopping, dropout, weight decay, or data augmentation.
- (3) No — on easy clean data classical ML is already near-perfect; CNNs win on hard, varied, large-scale data and via transfer learning.
- (4) Gradients from every batch accumulate, so each update uses a corrupted, oversized gradient.

Key idea

Match the tool to the problem — and remember the loop's four steps and their pitfalls.

Wrap-up

What you should now be able to do

- Explain a CNN as convolution with *learned* filters, plus ReLU, pooling, and a classifier.
- Describe how training works: forward, loss, backprop, gradient descent — with numbers.
- Read training curves and recognize the role of batches, epochs, learning rate, regularization.
- Judge honestly when a CNN beats classical ML — and when it doesn't.

This week's deliverable

Due before Week 14 (see `project/`)

1. Train the small digit CNN; report test accuracy and show the training curve.
2. Visualize the learned first-layer filters; describe what they detect.
3. Compare to a classical baseline (SVM/k-NN) on the same digits.
4. Add one regularizer (dropout or augmentation) and report its effect on validation accuracy.
5. Reflect: on your data, when would you expect a CNN to beat classical features?

A CNN learns the filters Week 3 made us design.

Convolution + ReLU + pooling, trained end to end — features and classifier together.